Incident Reporting & Management System

Implementation Plan & Technical Architecture

VERSION	DATE	PLATFORM	FRAMEWORK
1.0	July 2026	Android & iOS	Flutter + Node.js

Table of Contents

01 Executive Summary

02 System Workflow

03 Technology Stack

04 System Architecture

05 Feature Specification

06 Data Model

07 Project Structure

08 API Design

09 Security Architecture

10 Development Roadmap

11 Resource & Cost Estimates

12 Testing & QA Strategy

13 Deployment Strategy

SECTION 01

Executive Summary

The **Incident Reporting & Management System (IRMS)** is a cross-platform mobile application designed to enable community members to report incidents in real-time and route them to dispatchers for verification, assessment, and action. The system bridges the gap between public awareness and coordinated response by providing a streamlined digital pipeline from report to resolution.

2	4	4
PLATFORMS	USER ROLES	DEV PHASES

🎯 **PROJECT GOAL**

Build a production-ready, community-driven incident reporting platform that empowers citizens to report incidents and enables dispatchers to efficiently manage, prioritize, and resolve them — available on both Android and iOS from a single Flutter codebase.

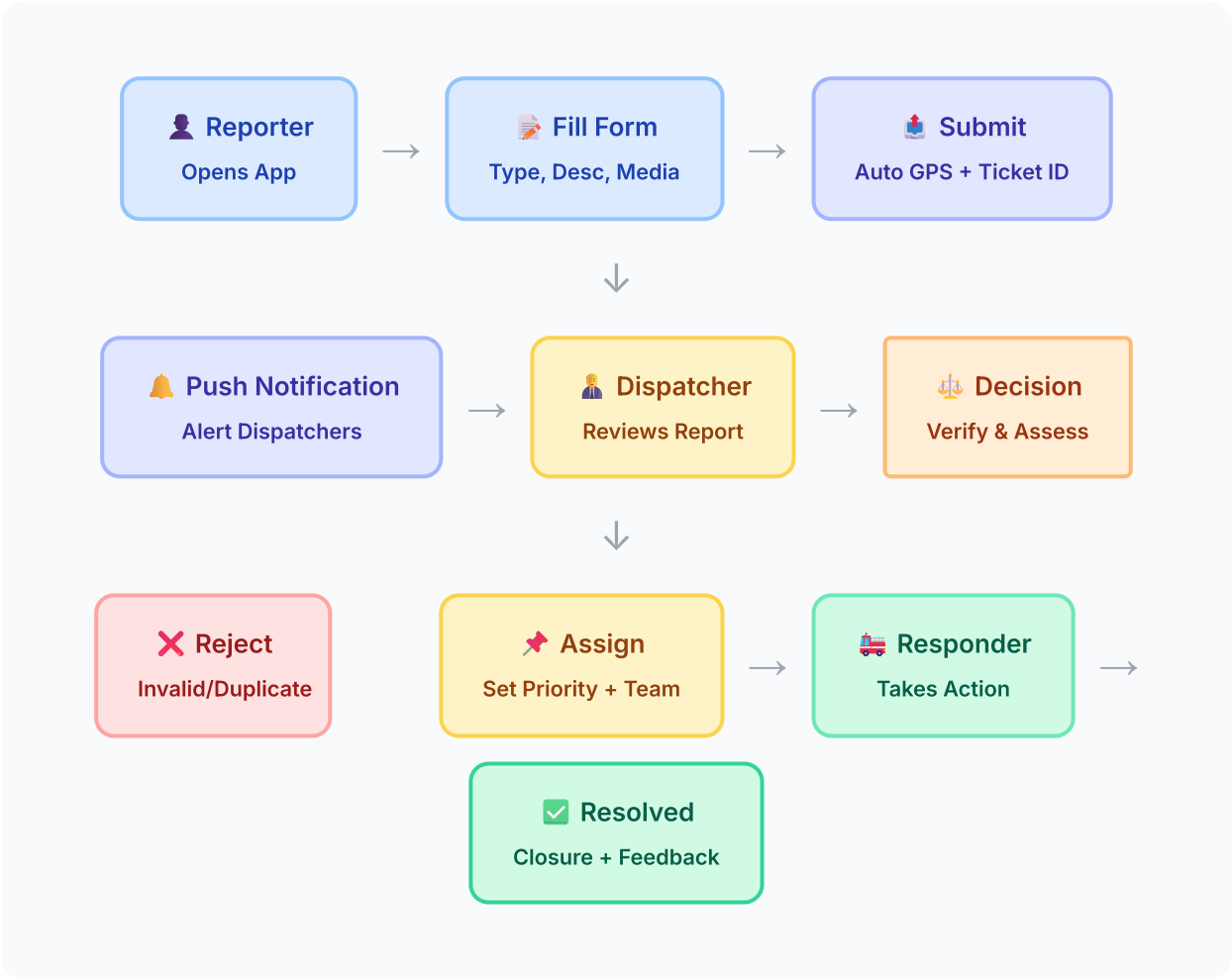
Key Stakeholders

ROLE	DESCRIPTION	PRIMARY ACTIONS
Reporter (General Public)	Community members who witness and report incidents	Submit reports, attach media, track status
Dispatcher	Trained personnel who triage and manage incoming reports	Verify, classify, prioritize, assign, communicate
Responder	Field personnel assigned to handle incidents	Accept assignments, provide updates, resolve
Administrator	System administrators managing users and configurations	User management, system config, analytics, auditing

SECTION 02

System Workflow

The core system workflow follows a linear pipeline from incident report submission to resolution, with decision points and feedback loops at each stage.

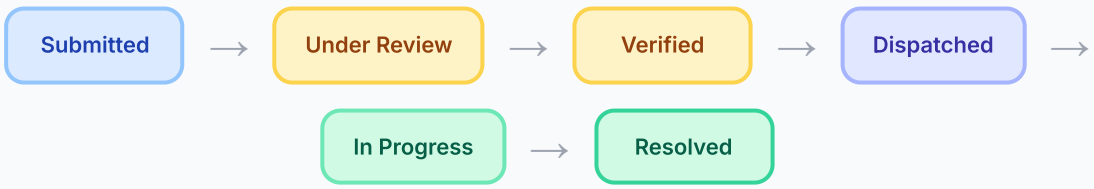


Detailed Process Flow

STEP	ACTOR	ACTION	SYSTEM RESPONSE
1	Reporter	Opens app, selects "Report Incident"	Presents categorized incident form
2	Reporter	Fills in type, description, severity, attaches media	Auto-captures GPS coordinates, validates fields
3	Reporter	Submits report	Generates unique ticket ID, sends confirmation push

STEP	ACTOR	ACTION	SYSTEM RESPONSE
4	System	Routes report to dispatcher queue	Push notification to all available dispatchers
5	Dispatcher	Reviews report: details, media, location on map	Displays full report with interactive map view
6	Dispatcher	Verifies legitimacy, sets severity & priority	Updates status to "Verified" or "Rejected"
7	Dispatcher	Assigns to responder team	Notifies assigned responders, updates status to "Dispatched"
8	Responder	Accepts assignment, provides progress updates	Status updates visible to dispatcher and reporter
9	Responder	Marks incident as resolved with notes	Closure notification sent to reporter
10	Reporter	Receives resolution, optionally rates response	Feedback stored for analytics and quality tracking

Incident Status Lifecycle



SECTION 03

Technology Stack

3.1 Frontend — Flutter (Dart)

✅ **SELECTED TECHNOLOGY**

Flutter 3.x with Dart — Single codebase for Android & iOS with native performance, a rich widget library, and excellent developer experience.

COMPONENT	TECHNOLOGY	PURPOSE
UI Framework	Flutter 3.x + Material Design 3	Cross-platform UI rendering
State Management	Riverpod / BLoC	Predictable state management
Navigation	GoRouter	Declarative routing with deep links
HTTP Client	Dio	REST API communication with interceptors
Local Storage	Hive / SharedPreferences	Offline data persistence
Maps	Google Maps Flutter / Mapbox	Interactive incident maps
Media	image_picker, camera, video_player	Capture and display media
Push Notifications	Firebase Cloud Messaging (FCM)	Real-time alerts
Geolocation	geolocator + geocoding	GPS capture and reverse geocoding

3.2 Backend — Node.js

✅ **SELECTED TECHNOLOGY**

Node.js with Express.js/Fastify — Full control over business logic, scalable REST API, WebSocket support for real-time features, no vendor lock-in.

COMPONENT	TECHNOLOGY	PURPOSE
Runtime	Node.js 20 LTS	Server-side JavaScript runtime
Framework	Express.js or Fastify	HTTP server and routing
ORM	Prisma	Type-safe database access
Validation	Zod / Joi	Request validation schemas
Authentication	JWT (jsonwebtoken) + bcrypt	Token-based auth with password hashing
Real-time	Socket.IO	WebSocket connections for live updates
File Upload	Multer + AWS S3 SDK	Media upload handling and storage
Email	Nodemailer / SendGrid	Email notifications
Task Queue	BullMQ + Redis	Background jobs (notifications, reports)

3.3 Database & Storage

LAYER	TECHNOLOGY	PURPOSE
Primary Database	PostgreSQL 16	Relational data: users, incidents, actions
Caching	Redis 7	Session cache, rate limiting, job queues
Object Storage	AWS S3 / Cloudflare R2	Photos, videos, audio files
Search (Phase 3)	Elasticsearch / Meilisearch	Full-text search across incidents

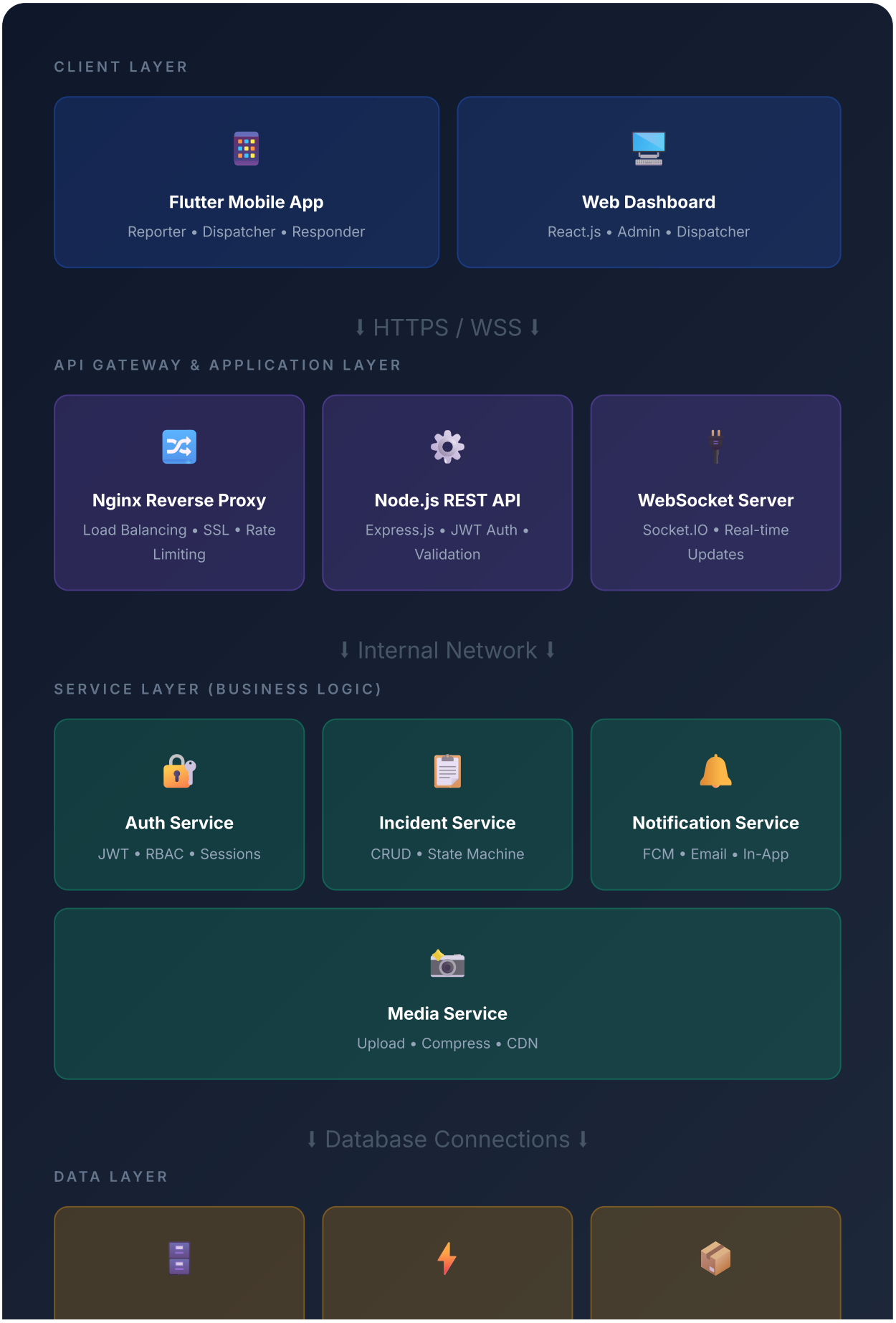
3.4 Infrastructure & DevOps

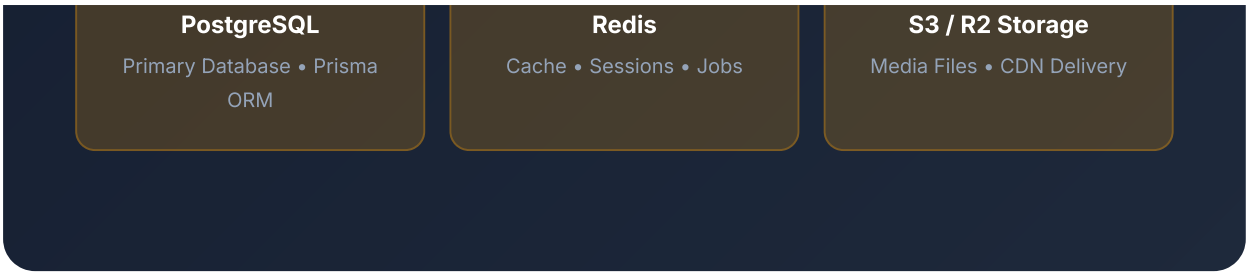
SERVICE	TECHNOLOGY	PURPOSE
Cloud Provider	AWS / Google Cloud / DigitalOcean	Server hosting, managed services
Containerization	Docker + Docker Compose	Consistent development & deployment

SERVICE	TECHNOLOGY	PURPOSE
CI/CD	GitHub Actions + Fastlane	Automated testing, building, and deployment
Monitoring	Sentry + Grafana	Error tracking, performance monitoring
Push Notifications	Firebase Cloud Messaging	Cross-platform push delivery
API Documentation	Swagger / OpenAPI	Interactive API docs

SECTION 04

System Architecture




PostgreSQL
Primary Database • Prisma ORM
Redis
Cache • Sessions • Jobs
S3 / R2 Storage
Media Files • CDN Delivery

SECTION 05

Feature Specification

5.1 Reporter Features (Mobile App)

FEATURE	PRIORITY	PHASE	DESCRIPTION
User Registration & Login	CRITICAL	1	Email/phone sign-up, login, password reset, profile management
Incident Submission Form	CRITICAL	1	Category selector, description, severity self-assessment, submit button
GPS Auto-Location	CRITICAL	1	Automatic GPS capture with option to manually adjust pin on map
Photo Attachment	CRITICAL	1	Camera capture or gallery upload, up to 5 photos per report
Status Tracking	CRITICAL	1	Real-time status updates: Submitted → Under Review → Dispatched → Resolved
Push Notifications	HIGH	1	Receive alerts when report status changes
Incident History	HIGH	1	View list of past submitted reports with status
Video & Audio Attachment	HIGH	2	Record and attach video (up to 60s) and voice memos
In-App Messaging	HIGH	2	Chat with dispatcher for follow-up questions or updates
Anonymous Reporting	MEDIUM	3	Submit reports without revealing identity
Offline Mode	MEDIUM	3	Queue reports when offline, auto-submit when connection restores
SOS Emergency Button	MEDIUM	3	One-tap emergency report with auto-location and pre-filled data
Community Feed	LOW	4	Browse anonymized nearby active/resolved incidents on map

5.2 Dispatcher Features (Mobile + Web Dashboard)

FEATURE	PRIORITY	PHASE	DESCRIPTION
Live Incident Queue	CRITICAL	1	Real-time feed of incoming reports, sortable by time/severity/status
Report Detail View	CRITICAL	1	Full report with all media, map location, reporter info, timeline
Verify & Classify	CRITICAL	1	Mark valid/invalid, set category, severity level, add dispatcher notes
Assign to Responder	CRITICAL	1	Select and assign incident to available responder or team
Interactive Map View	HIGH	2	All active incidents plotted on map with clustering and filtering
Communication Channel	HIGH	2	In-app messaging with reporter and responder per incident
Analytics Dashboard	MEDIUM	3	Response time metrics, incident heatmaps, trend analysis
Auto-Escalation Rules	MEDIUM	3	Auto-escalate if unactioned after configurable time threshold
Bulk Operations	MEDIUM	3	Merge duplicates, bulk status updates, batch assignment

5.3 Admin Features (Web Dashboard)

FEATURE	PRIORITY	PHASE	DESCRIPTION
User Management	CRITICAL	1	Create, edit, deactivate users; assign roles
Role-Based Access Control	CRITICAL	1	Granular permissions per role (reporter, dispatcher, responder, admin)
Incident Category Config	HIGH	2	Customize incident types, severity levels, and form fields

FEATURE	PRIORITY	PHASE	DESCRIPTION
Report Export	HIGH	2	Export incident data as PDF/CSV for reporting
Audit Logs	MEDIUM	3	Immutable log of all user actions for accountability
System Configuration	MEDIUM	3	Notification rules, assignment logic, data retention settings

SECTION 06

Data Model

The core database schema consists of five primary entities with well-defined relationships.

USER

id PK	UUID	Auto-generated unique identifier
name	VARCHAR(255)	Full name
email	VARCHAR(255)	Unique email address
phone	VARCHAR(20)	Contact phone number
password_hash	VARCHAR(255)	bcrypt hashed password
role	ENUM	reporter | dispatcher | responder | admin
avatar_url	TEXT	Profile photo URL
is_active	BOOLEAN	Account active status
created_at	TIMESTAMP	Account creation time
updated_at	TIMESTAMP	Last update time

🚨 INCIDENT

id PK	UUID	Auto-generated unique identifier
ticket_number	VARCHAR(20)	Human-readable ticket (e.g., INC-2026-00001)
reporter_id FK	UUID	References USER.id (nullable for anonymous)
dispatcher_id FK	UUID	Assigned dispatcher (nullable until reviewed)
responder_id FK	UUID	Assigned responder (nullable until dispatched)
type	ENUM	fire | accident | crime | medical | natural_disaster | infrastructure | other
title	VARCHAR(255)	Brief incident title
description	TEXT	Detailed incident description
severity	ENUM	low | medium | high | critical
status	ENUM	submitted | under_review | verified | dispatched | in_progress | resolved | rejected
latitude	DECIMAL(10,8)	GPS latitude coordinate
longitude	DECIMAL(11,8)	GPS longitude coordinate
address	TEXT	Reverse-geocoded address string
is_anonymous	BOOLEAN	Whether reporter identity is hidden
dispatcher_notes	TEXT	Internal notes from dispatcher
resolution_notes	TEXT	Notes upon resolution
created_at	TIMESTAMP	Report submission time
updated_at	TIMESTAMP	Last update time
resolved_at	TIMESTAMP	Resolution time (nullable)

📷 MEDIA

id PK	UUID	Auto-generated unique identifier
incident_id FK	UUID	References INCIDENT.id
type	ENUM	photo | video | audio
url	TEXT	S3/R2 URL for the media file
thumbnail_url	TEXT	Compressed thumbnail URL
file_size	INTEGER	File size in bytes
created_at	TIMESTAMP	Upload time

📝 ACTION_LOG

id PK	UUID	Auto-generated unique identifier
incident_id FK	UUID	References INCIDENT.id
actor_id FK	UUID	References USER.id (who performed the action)
action	ENUM	created | verified | rejected | assigned | escalated | resolved | commented
old_status	ENUM	Previous status (nullable)
new_status	ENUM	New status after action
notes	TEXT	Action notes / comments
created_at	TIMESTAMP	Action time (immutable)

🔔 NOTIFICATION

id PK	UUID	Auto-generated unique identifier
user_id FK	UUID	Target user
incident_id FK	UUID	Related incident (nullable)
title	VARCHAR(255)	Notification title
body	TEXT	Notification body text
is_read	BOOLEAN	Read status (default false)
created_at	TIMESTAMP	Notification creation time

Entity Relationships

RELATIONSHIP	TYPE	DESCRIPTION
USER → INCIDENT (reporter)	One-to-Many	A user can report many incidents
USER → INCIDENT (dispatcher)	One-to-Many	A dispatcher can manage many incidents
USER → INCIDENT (responder)	One-to-Many	A responder can be assigned many incidents
INCIDENT → MEDIA	One-to-Many	An incident can have multiple media files
INCIDENT → ACTION_LOG	One-to-Many	An incident tracks all actions taken
USER → NOTIFICATION	One-to-Many	A user receives many notifications

RELATIONSHIP	TYPE	DESCRIPTION
INCIDENT → NOTIFICATION	One-to-Many	An incident can trigger many notifications

SECTION 07

Project Structure

7.1 Flutter App Structure

```
irms_app/
├── lib/
│   ├── main.dart                    # App entry point
│   ├── config/
│   │   ├── app_config.dart          # Environment config
│   │   ├── routes.dart              # GoRouter config
│   │   └── theme.dart               # Material 3 theme
│   ├── core/
│   │   ├── network/
│   │   │   ├── api_client.dart      # Dio HTTP client
│   │   │   ├── api_endpoints.dart   # Endpoint constants
│   │   │   └── socket_service.dart  # Socket.IO client
│   │   ├── storage/
│   │   │   └── local_storage.dart   # Hive/SharedPrefs
│   │   └── utils/
│   │       ├── location_utils.dart  # GPS helpers
│   │       └── media_utils.dart     # Image compression
│   ├── features/
│   │   ├── auth/
│   │   │   ├── models/              # User model
│   │   │   ├── providers/           # Auth state (Riverpod)
│   │   │   ├── repositories/        # Auth API calls
│   │   │   └── screens/             # Login, Register, Forgot
│   │   ├── incidents/
│   │   │   ├── models/              # Incident, Media models
│   │   │   ├── providers/           # Incident state
│   │   │   ├── repositories/        # Incident API calls
│   │   │   └── screens/
│   │   │       ├── report_form.dart     # Reporter submit form
│   │   │       ├── incident_list.dart   # History / queue view
│   │   │       ├── incident_detail.dart# Full report view
│   │   │       └── map_view.dart        # Map with incidents
│   │   ├── dispatcher/
│   │   │   ├── providers/           # Dispatcher queue state
│   │   │   └── screens/
│   │   │       ├── dispatch_queue.dart# Live incident queue
│   │   │       ├── review_screen.dart   # Verify & classify
│   │   │       └── assign_screen.dart   # Assign responders
│   │   └── notifications/
│   │       ├── providers/           # Notification state
│   │       └── screens/             # Notification list
│   └── shared/
│       ├── widgets/                 # Reusable UI components
│       └── models/                  # Shared data models
├── assets/                          # Icons, images, fonts
├── test/                            # Unit & widget tests
├── integration_test/                # Integration tests
└── pubspec.yaml                     # Dependencies
```

7.2 Node.js Backend Structure

```
irms_api/
├── src/
│   ├── app.js                       # Express app setup
│   ├── server.js                    # HTTP + WebSocket server
│   ├── config/
│   │   ├── database.js              # PostgreSQL connection
│   │   ├── redis.js                 # Redis connection
│   │   └── s3.js                    # S3/R2 configuration
│   ├── middleware/
│   │   ├── auth.js                  # JWT verification
│   │   ├── rbac.js                  # Role-based access
│   │   ├── validate.js              # Request validation
│   │   ├── upload.js                # Multer config
│   │   └── rateLimiter.js           # Rate limiting
│   ├── routes/
│   │   ├── auth.routes.js           # /api/auth/*
│   │   ├── incident.routes.js       # /api/incidents/*
│   │   ├── user.routes.js           # /api/users/*
│   │   ├── notification.routes.js   # /api/notifications/*
│   │   └── media.routes.js          # /api/media/*
│   ├── controllers/                 #
```

```
Route handlers | ├── services/ # Business logic | ├── models/ # Prisma-generated models | ├── validators/ # Zod schemas | ├── sockets/ | | └── incident.socket.js # Real-time incident events | └── jobs/ | ├── notification.job.js # Push notification queue | └── escalation.job.js # Auto-escalation checker ├── prisma/ | ├── schema.prisma # Database schema | └── migrations/ # Migration files ├── tests/ # Jest test suites ├── Dockerfile # Container config ├── docker-compose.yml # Full stack compose ├── .env.example # Environment template └── package.json # Dependencies
```

SECTION 08

API Design

RESTful API design following industry best practices with consistent response formats and proper HTTP status codes.

8.1 Authentication Endpoints

METHOD	ENDPOINT	DESCRIPTION	AUTH
POST	/api/auth/register	Create new user account	No
POST	/api/auth/login	Login with credentials	No
POST	/api/auth/refresh	Refresh access token	Refresh Token
POST	/api/auth/forgot-password	Request password reset	No
POST	/api/auth/reset-password	Reset password with token	Reset Token
POST	/api/auth/logout	Invalidate current session	JWT

8.2 Incident Endpoints

METHOD	ENDPOINT	DESCRIPTION	ROLES
POST	/api/incidents	Create new incident report	Reporter
GET	/api/incidents	List incidents (filtered by role)	All
GET	/api/incidents/:id	Get incident details	All
PATCH	/api/incidents/:id/status	Update incident status	Dispatcher, Responder
PATCH	/api/incidents/:id/assign	Assign incident to responder	Dispatcher

METHOD	ENDPOINT	DESCRIPTION	ROLES
PATCH	`/api/incidents/:id/verify`	Verify/reject incident	Dispatcher
GET	`/api/incidents/:id/timeline`	Get incident action log	All
GET	`/api/incidents/map`	Get incidents for map view	Dispatcher

8.3 Media Endpoints

METHOD	ENDPOINT	DESCRIPTION	ROLES
POST	`/api/media/upload`	Upload media file(s)	Reporter
GET	`/api/media/:id`	Get media signed URL	All
DELETE	`/api/media/:id`	Delete media file	Admin

8.4 User & Notification Endpoints

METHOD	ENDPOINT	DESCRIPTION	ROLES
GET	`/api/users/me`	Get current user profile	All
PATCH	`/api/users/me`	Update own profile	All
GET	`/api/users`	List all users	Admin
PATCH	`/api/users/:id/role`	Update user role	Admin
GET	`/api/notifications`	List user notifications	All
PATCH	`/api/notifications/:id/read`	Mark notification as read	All
POST	`/api/users/me/fcm-token`	Register FCM device token	All

8.5 WebSocket Events

EVENT	DIRECTION	DESCRIPTION
`incident:new`	Server → Client	New incident submitted (broadcasted to dispatchers)
`incident:updated`	Server → Client	Incident status or details changed
`incident:assigned`	Server → Client	Incident assigned to a responder
`incident:resolved`	Server → Client	Incident has been resolved
`message:new`	Bidirectional	New chat message on incident thread
`notification:new`	Server → Client	New notification for the user

SECTION 09

Security Architecture

SECURITY LAYER	IMPLEMENTATION	DETAILS
Authentication	JWT with Refresh Tokens	Short-lived access tokens (15min), long-lived refresh tokens (7d) with rotation. Stored securely using Flutter Secure Storage.
Authorization	Role-Based Access Control	Middleware-enforced RBAC on every API endpoint. Roles: reporter, dispatcher, responder, admin. Row-level security on user data.
Transport Security	TLS 1.3	All communications over HTTPS. Certificate pinning in the Flutter app for production builds.
Data at Rest	AES-256 Encryption	Database encryption via PostgreSQL pgcrypto. S3 server-side encryption for media files.
Password Security	bcrypt (12 rounds)	Salted password hashing. Enforce minimum 8 chars, complexity requirements.
Input Validation	Zod Schemas	Server-side validation for all inputs. Sanitize HTML/scripts. Validate file types and sizes.
Rate Limiting	Redis-backed Limiter	Per-user: 100 req/min. Per-IP: 200 req/min. Login: 5 attempts/15min. Report submission: 10/hour.
Media Privacy	Signed URLs	Pre-signed S3 URLs with 1-hour expiration. No public bucket access. Strip EXIF metadata.
Anonymous Reports	Detached Identity	Anonymous reports use a one-time token. No PII linked. Reporter cannot be traced.
Audit Trail	Immutable Action Logs	All state changes logged with actor, timestamp, old/new values. Append-only (no UPDATE/DELETE).
CORS & CSP	Strict Policies	Whitelist allowed origins. Content Security Policy headers on web dashboard.

⚠ **PRIVACY COMPLIANCE**

Implement data retention policies (auto-delete resolved incidents after configurable period), user data export (GDPR Article 15), and account deletion (GDPR Article 17). Display clear

privacy policy and terms of service in the app.

SECTION 10

Development Roadmap

1 Phase 1 — MVP 6 – 8 Weeks

- Project scaffolding: Flutter app + Node.js API + PostgreSQL + Docker
- User authentication: Registration, login, JWT token flow
- Incident submission form with category, description, photo, GPS
- Dispatcher queue: Live list of submitted reports
- Report detail view with media and map location
- Verify / Reject / Assign workflow for dispatchers
- Push notifications via Firebase Cloud Messaging
- Reporter status tracking (timeline view)
- Basic user management (admin)
- Core unit & integration tests

2 Phase 2 — Core Features 4 – 6 Weeks

- Video & audio media attachments with compression
- Interactive map view for dispatchers (incident clustering, filtering)
- Responder mobile workflow: accept assignment, update status
- In-app messaging (reporter ↔ dispatcher ↔ responder)
- Web dashboard for dispatchers & admins (React.js)
- Incident category & severity configuration (admin)
- Report export: PDF & CSV generation
- WebSocket real-time updates for queue & status changes

3 Phase 3 — Advanced Features 4 – 6 Weeks

- Analytics dashboard with incident heatmaps & response time metrics
- Offline mode with sync queue for reporters
- Auto-escalation rules (time-based, severity-based)
- Anonymous reporting with privacy-safe architecture
- Bulk operations: merge duplicates, batch status updates
- Comprehensive audit logging
- Full-text search across incidents (Meilisearch)

4 Phase 4 — Scale & Intelligence Ongoing

- AI-powered incident categorization & severity assessment
- Predictive analytics: hotspot prediction, resource planning
- Internationalization (i18n) & localization
- Accessibility (WCAG 2.1 AA compliance)
- Performance optimization & horizontal scaling
- Community feed: anonymized nearby incident map
- Third-party integrations (emergency services, government APIs)

SECTION 11

Resource & Cost Estimates

11.1 Team Requirements

ROLE	COUNT	RESPONSIBILITY	PHASES
Flutter Developer	1 – 2	Mobile app (all features, all screens)	All
Node.js Developer	1	REST API, WebSockets, database, services	All
UI/UX Designer	1	Wireframes, prototypes, design system	1 – 2
QA Engineer	1	Manual + automated testing	All
DevOps Engineer	0.5	CI/CD, cloud infra, monitoring	1, 4
Project Manager	0.5	Coordination, timeline, stakeholders	All

11.2 Infrastructure Costs (Monthly, Post-Launch)

SERVICE	FREE / DEV	GROWTH (~10K USERS)	SCALE (~100K USERS)
Compute (VPS/Cloud)	$0 (local)	$50 – $150	$300 – $1,000
PostgreSQL (Managed)	$0 (local)	$25 – $75	$100 – $400
Redis (Managed)	$0 (local)	$15 – $30	$50 – $150
Object Storage (S3/R2)	$0 (R2 free tier)	$10 – $40	$100 – $500
Push Notifications (FCM)	Free	Free	Free
Maps API	$200 credit/mo	$50 – $200	$500 – $2,000
Email (SendGrid)	Free (100/day)	$15 – $30	$50 – $100

SERVICE	FREE / DEV	GROWTH (~10K USERS)	SCALE (~100K USERS)
Monitoring (Sentry)	Free tier	$26	$80
Domain + SSL	$12/yr	$12/yr	$12/yr
Total Monthly	**~$0**	**~$190 – $550**	**~$1,180 – $4,230**

💡 COST OPTIMIZATION TIPS

Use **Cloudflare R2** instead of AWS S3 for zero egress costs. Use **Mapbox** free tier (50K loads/mo) instead of Google Maps for lower costs. Consider **DigitalOcean** or **Hetzner** for 40-60% cheaper compute vs AWS/GCP for early stages.

SECTION 12

Testing & QA Strategy

TEST TYPE	TOOL	SCOPE	COVERAGE TARGET
Unit Tests (Flutter)	flutter_test	Models, providers, utils, business logic	80%+
Widget Tests (Flutter)	flutter_test	UI components, forms, interactions	70%+
Integration Tests (Flutter)	integration_test	Full user flows on real devices	Critical paths
Unit Tests (API)	Jest	Services, validators, middleware	85%+
API Integration Tests	Supertest + Jest	All endpoints with real DB	All endpoints
E2E Tests	Patrol (Flutter)	Full workflow: report → dispatch → resolve	Happy paths
Load Testing	k6 / Artillery	API under concurrent load	1000 concurrent users
Security Testing	OWASP ZAP	Vulnerability scanning	No critical/high findings

CI/CD PIPELINE

GitHub Actions will run linting, unit tests, and integration tests on every pull request. Merges to main trigger automated builds (APK/IPA via Fastlane) and deploy to staging environment for QA review.

SECTION 13

Deployment Strategy

13.1 Mobile App

PLATFORM	DISTRIBUTION	CI/CD TOOL	REQUIREMENTS
Android	Google Play Store	Fastlane + GitHub Actions	Google Play Developer Account ($25 one-time)
iOS	Apple App Store	Fastlane + GitHub Actions	Apple Developer Program ($99/year)
Beta Testing	TestFlight (iOS) / Internal Testing (Android)	Fastlane	Invite-based access for testers

13.2 Backend

ENVIRONMENT	PURPOSE	INFRASTRUCTURE
Development	Local development & testing	Docker Compose (API + PostgreSQL + Redis)
Staging	Pre-production testing & QA	DigitalOcean Droplet / AWS EC2 (small)
Production	Live user-facing system	DigitalOcean App Platform / AWS ECS / Railway

13.3 Environment Configuration

🔧 **ENVIRONMENT VARIABLES**

All sensitive configuration (database URLs, API keys, JWT secrets) must be stored as environment variables. Never commit secrets to version control. Use a `.env.example` file as a template.

VARIABLE	DESCRIPTION	EXAMPLE
DATABASE_URL	PostgreSQL connection string	postgresql://user:pass@host:5432/irms
REDIS_URL	Redis connection string	redis://host:6379
JWT_SECRET	JWT signing secret	[random 64-char string]
JWT_REFRESH_SECRET	Refresh token signing secret	[random 64-char string]
S3_BUCKET	Object storage bucket name	irms-media
S3_ACCESS_KEY	S3/R2 access key	[access key]
S3_SECRET_KEY	S3/R2 secret key	[secret key]
FCM_SERVER_KEY	Firebase Cloud Messaging key	[FCM key]
GOOGLE_MAPS_API_KEY	Google Maps API key	[API key]